



Faculty of Engineering & Technology
Electrical & Computer Engineering Department
Artificial Intelligence - ENCS3340

Project 1

Optimizing Hospital Locations Using Search and Optimization Algorithms

Prepared by:

Ahmad Khdair 1230500

Malek Zeghari 1230358

Instructor: Ismail Khater

Section: 3

Date: June 5, 2026

Abstract

This report presents a comparative study of Hill Climbing and Simulated Annealing for optimizing hospital locations. The task is to choose a subset of candidate locations that minimizes weighted travel distance while penalizing the number of built hospitals. The experiments use the baseline instance required in the project: 100 population points, 100 candidate sites, Euclidean distance, and lambda values 1, 10, 50, and 100.

The results show that both algorithms correctly reflect the lambda trade-off: increasing lambda reduces the number of selected hospitals and increases travel distance. Hill Climbing produced lower or equal mean cost in all tested settings and had much lower variance and runtime. Simulated Annealing was more exploratory but more sensitive to its cooling parameter and showed higher instability under the fixed evaluation budget.

The report is organized as follows: Section 1 introduces the problem, Section 2 formulates the objective and state representation, Section 3 explains the algorithms, Section 4 describes the experimental setup, Sections 5-8 present results, plots, visualizations, and tuning, and Sections 9-11 discuss the findings and conclude.

Report Contributions

- Formalized the hospital-location problem as a local-search optimization problem.
- Implemented and compared Hill Climbing and Simulated Annealing under the same evaluation budget.
- Studied the effect of lambda on hospital count and patient travel distance.
- Tuned one important parameter for each algorithm.
- Visualized the selected hospital locations for each lambda value.

1. Introduction

The goal of this project is to select where hospitals should be built from a fixed set of candidate locations. The selected hospitals should reduce the weighted travel distance for the population, but the solution should not open too many hospitals because each hospital has a construction penalty controlled by lambda.

This makes the problem a natural local-search optimization problem: the path used to reach a solution is not important; what matters is the final subset of selected candidate locations and its objective value. The report compares Hill Climbing and Simulated Annealing on the same generated instance using the required lambda values and repeated runs.

The main questions studied are: How does lambda affect the number of hospitals? Which algorithm gives lower final cost? Which algorithm is more stable across runs? How do algorithm parameters such as initialization rate and cooling rate affect the results?

2. Problem Formulation

Given population points $P = \{p_1, p_2, \dots, p_n\}$, population weights $W = \{w_1, w_2, \dots, w_n\}$, and candidate hospital sites $C = \{c_1, c_2, \dots, c_m\}$, a solution H is any non-empty subset of C . Each population point is assigned to its nearest hospital in H using Euclidean distance.

The objective function minimized in all experiments is:

$$\text{TotalCost}(H) = \sum_{i=1}^n w_i D_i(H) + \lambda |H|$$
$$D_i(H) = \min_{h \in H} \text{dist}(p_i, h)$$

The first term represents weighted accessibility cost. The second term represents the construction penalty. A small lambda makes it cheap to open hospitals, so the algorithms tend to select many sites. A large lambda makes hospitals expensive, so the algorithms accept larger travel distances in exchange for fewer hospitals.

Parameter	Value
Population points (n)	100
Candidate sites (m)	100
Coordinate space	[0, 100] x [0, 100]
Population weights	Uniform integers in [1, 10]
Distance metric	Euclidean distance
Lambda values	1, 10, 50, 100
Random seed	7

Table 1: Baseline problem parameters.

2.1 Search Representation

The state is represented as a binary vector of length m, where 1 means that the corresponding candidate site is selected as a hospital, and 0 means it is not selected. This representation is compact and directly supports local modifications.

Component	Description
State	Binary vector over the 100 candidate sites
Initial state	Random non-empty subset of candidate sites
Action / Neighbor	Flip one bit: add one hospital or remove one hospital
Goal	A low-cost subset according to the objective function
Validity rule	At least one hospital must remain selected

Table 2: Search formulation used by both algorithms.

3. Algorithm Descriptions

3.1 Hill Climbing

Hill Climbing is used as a greedy local search method. Starting from a random solution, it evaluates all neighbors created by flipping one candidate site. The algorithm then moves to the best improving neighbor. Since the implementation minimizes cost, this is equivalent to moving downhill in the objective landscape.

- **Initialization:** random solution with around 10-20% of candidates selected.
- **Neighborhood:** all one-bit flips, meaning adding or removing exactly one hospital.
- **Move rule:** best-improvement; choose the neighbor with the lowest cost if it improves the current state.
- **Stopping condition:** stop at a local optimum, at 500 iterations, or when the evaluation budget is reached.
- **Strength:** uses evaluations efficiently because it compares all local alternatives before moving.
- **Weakness:** can stop at a local optimum that is not globally optimal.

3.2 Simulated Annealing

Simulated Annealing is a probabilistic local search method designed to reduce the risk of getting trapped in local optima. At each step, it randomly selects a neighbor. Better neighbors are accepted, while worse neighbors may also be accepted depending on the temperature. As the temperature decreases, the algorithm becomes more greedy.

The cooling schedule used in the main experiment is: $T(k) = \max(T_0 \cdot \alpha^k, T_{\min})$, with $T_0 = 1000$, $\alpha = 0.95$, and $T_{\min} = 1e-10$.

- **Initialization:** random non-empty solution.
- **Neighborhood:** random one-bit flip.
- **Acceptance rule:** accept improving moves; sometimes accept worse moves with probability based on cost increase and temperature.
- **Stopping condition:** 20,000 evaluations or cooling to the minimum temperature.
- **Best-state memory:** the algorithm returns the best solution found during the run, not necessarily the last visited state.
- **Strength:** can escape some local optima by temporarily accepting worse moves.
- **Weakness:** sensitive to the cooling schedule and more stochastic across runs.

3.3 Fairness of Comparison

Both algorithms use the same objective function, the same candidate and population points, and the same maximum evaluation budget of 20,000 per run. This makes the comparison mainly about how each algorithm uses its evaluations, not about giving one algorithm more computational opportunity.

The comparison is based on objective-function evaluations rather than raw loop iterations. This is fairer because one Hill Climbing iteration can examine many one-bit-flip neighbors, while one Simulated Annealing step usually evaluates only one sampled neighbor. Therefore, evaluation budget measures the actual number of cost computations allowed for each algorithm.

3.4 Implementation Edge Cases

The implementation guarantees that every solution remains non-empty. If a candidate flip would remove the last selected hospital, the move is skipped using continue before evaluating the invalid neighbor. In Simulated Annealing, there is also a special case for $m == 1$: if there is only one candidate site, that site must be selected and the algorithm returns immediately. These checks avoid the invalid case $H = \text{empty set}$, where every population point would have no nearest hospital.

4. Experimental Setup

Setting	Value
Programming language	Python 3.12
Libraries	NumPy and Matplotlib
Runs per lambda per algorithm	10 independent runs
Evaluation budget	20,000 objective evaluations per run
Hill Climbing max iterations	500
HC initial selection rate	Random 10-20% in main runs
SA initial temperature	1000

SA cooling rate	0.95
-----------------	------

Table 3: Experimental configuration.

The generated problem instance follows the suggested baseline setup: 100 uniformly distributed population points, 100 uniformly distributed candidate hospital locations, and integer population weights between 1 and 10. The same distance matrix is reused across all algorithm runs.

4.1 Evaluation Metrics

Metric	Meaning
Mean cost	Average objective value over repeated runs
Best cost	Lowest objective value found across repeated runs
Variance	Stability indicator; lower variance means more consistent results
Mean hospitals	Average number of selected hospital sites
Mean average travel	Weighted average distance from population points to nearest hospital
Mean runtime	Average wall-clock time per run
Mean evaluations	Average number of objective-function evaluations used

Table 4: Metrics used to evaluate the algorithms.

4.2 Program Input and Output

The implementation can work in two modes. In random-generation mode, it creates the population points, weights, and candidate sites using the configured random seed. In CSV-input mode, the population file contains x, y, and weight columns, while the candidate-sites file contains x and y columns.

For each experiment, the program records one row per run containing the algorithm name, lambda value, run number, total cost, runtime, number of hospitals, average travel distance, and number of objective evaluations. It also creates summary CSV files and plots used in this report.

4.3 Reproducibility Notes

A fixed random seed is used to make the generated instance reproducible. Separate seed offsets are used for Hill Climbing and Simulated Annealing so that the two algorithms do not receive identical random streams, while still running on the same underlying problem instance.

5. Main Experimental Results

Algorithm	λ	Mean Cost	Best Cost	Variance	Hosp.	Avg. Travel	Time(s)	Evals
Hill Climbing	1	3167.56	3167.56	0.00	60.0	5.63	0.243	5971
Sim. Annealing	1	3167.56	3167.56	0.00	60.0	5.63	1.352	20000
Hill Climbing	10	3663.96	3656.34	29.42	50.1	5.73	0.175	4961
Sim. Annealing	10	3678.36	3664.34	59.54	50.2	5.75	1.123	20000
Hill Climbing	50	5132.87	5075.04	1595.52	27.1	6.84	0.101	2861
Sim. Annealing	50	5204.56	5100.02	3699.97	28.4	6.86	1.330	20000
Hill Climbing	100	6351.38	6292.33	2351.17	20.9	7.72	0.082	2461
Sim. Annealing	100	6449.73	6365.40	5803.86	19.7	8.12	1.002	20000

Table 5: Main results averaged over 10 runs.

At $\lambda = 1$, both algorithms reach the same solution in all runs: 60 hospitals, an average weighted travel distance of 5.63, and zero cost variance. For larger λ values, Hill Climbing obtains lower mean cost and lower variance than Simulated Annealing in this experiment.

5.1 Cost Decomposition

The total cost contains two parts: weighted travel cost and construction penalty. Since the total population weight in the generated instance is 552, the travel component can be read as 552 multiplied by the reported mean average travel distance. The table below makes the trade-off clearer for the mean solutions.

Algorithm	λ	Travel component	Construction penalty	Approx. total
Hill Climbing	1	3107.56	60.00	3167.56
Sim. Annealing	1	3107.56	60.00	3167.56
Hill Climbing	10	3162.96	501.00	3663.96
Sim. Annealing	10	3176.36	502.00	3678.36
Hill Climbing	50	3777.87	1355.00	5132.87
Sim. Annealing	50	3784.56	1420.00	5204.56
Hill Climbing	100	4261.38	2090.00	6351.38
Sim. Annealing	100	4479.73	1970.00	6449.73

Table 6: Approximate decomposition of mean total cost.

The decomposition shows why the travel distance rises with λ : reducing the construction penalty requires selecting fewer hospitals, but fewer hospitals make the travel component larger.

6. Result Plots and Interpretation

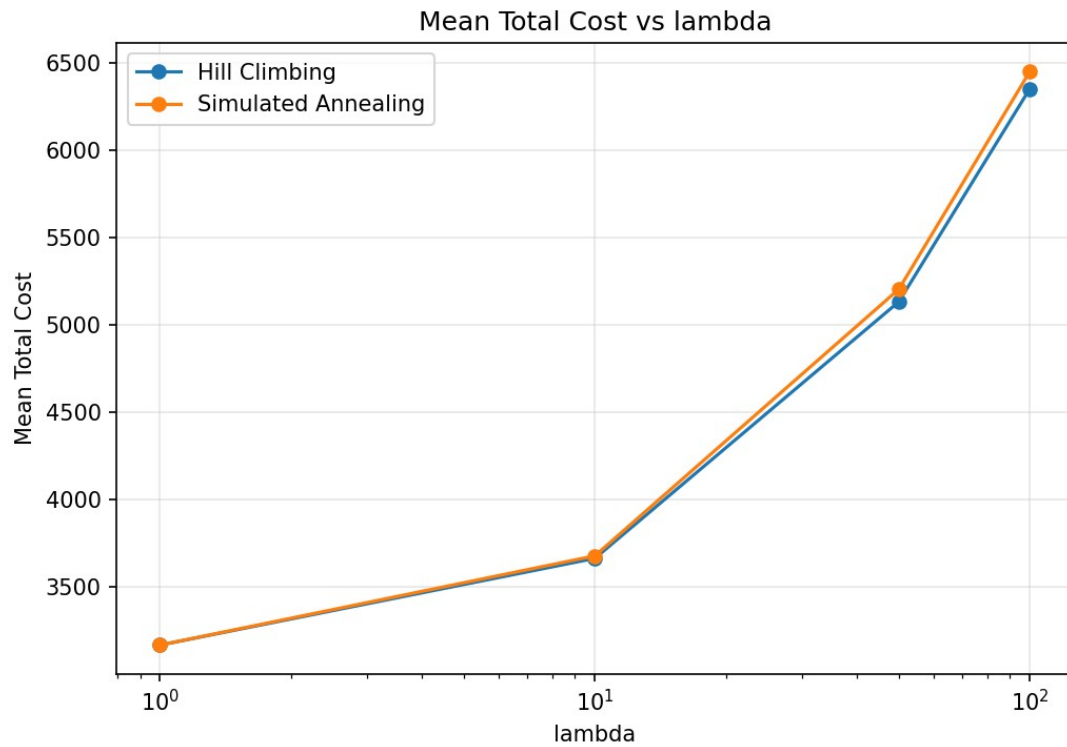


Figure 1: Mean total cost vs lambda.

Mean total cost increases as lambda increases. This is expected because the model makes each hospital more expensive, so the selected solution contains fewer hospitals and the remaining travel term increases.

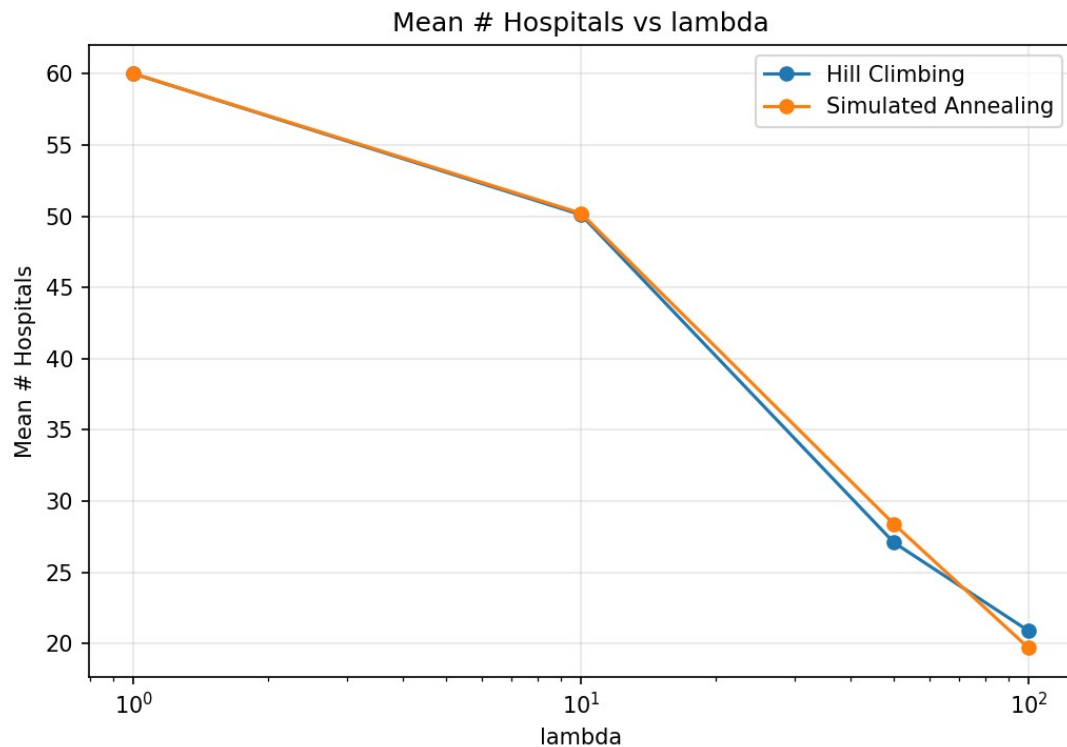


Figure 2: Mean number of hospitals vs lambda.

The number of selected hospitals decreases sharply as lambda grows. This validates the role of lambda as the parameter that controls the trade-off between accessibility and construction cost.

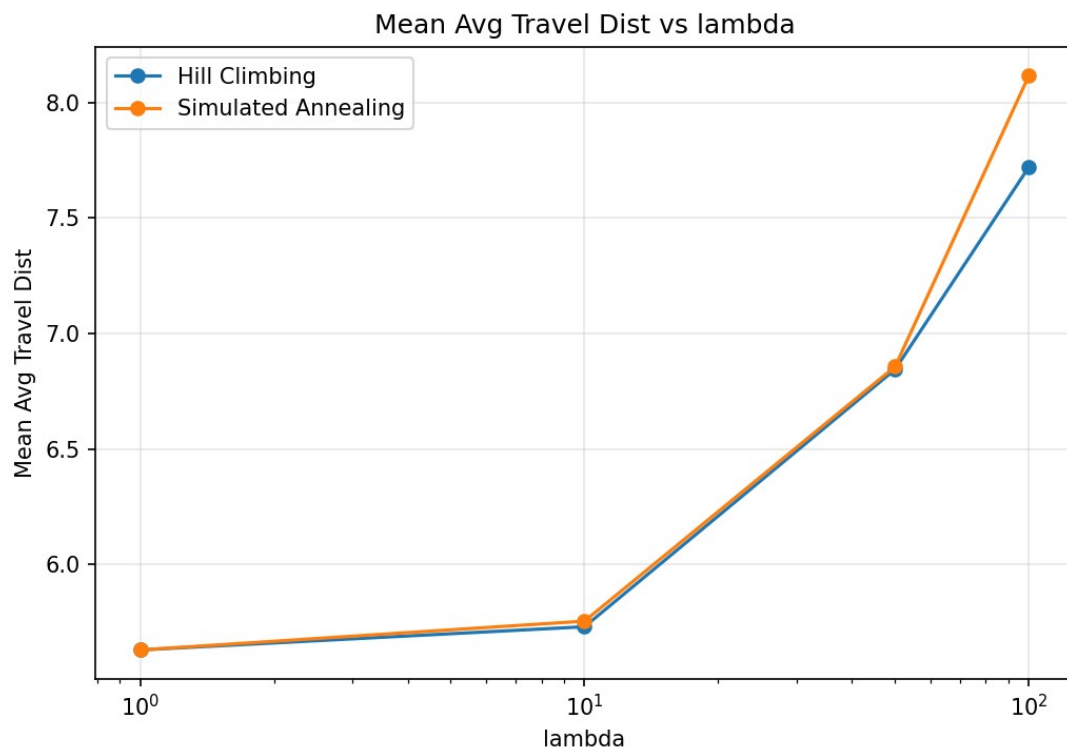


Figure 3: Mean weighted average travel distance vs lambda.

Average travel distance increases when fewer hospitals are opened. At $\lambda = 100$, each open hospital must serve a larger region, so patients are farther from their nearest selected site on average.

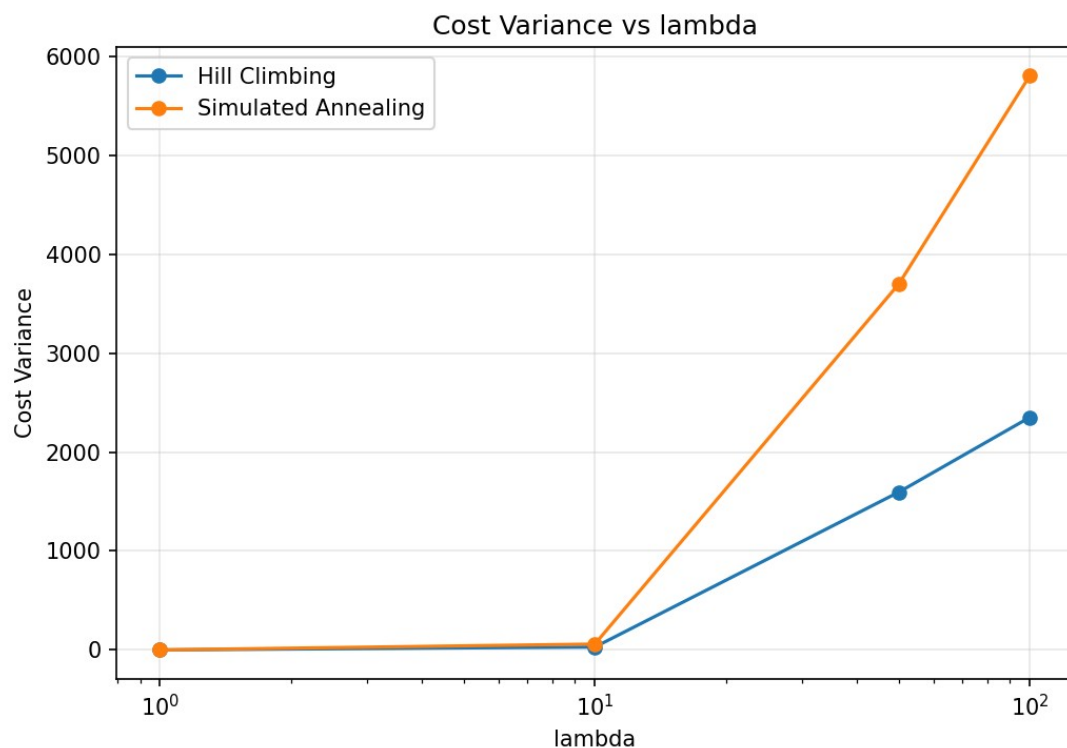


Figure 4: Cost variance vs lambda.

Hill Climbing is more stable in these runs. Simulated Annealing has higher variance, especially at $\lambda = 50$ and $\lambda = 100$, because it depends more heavily on random transitions and the cooling behavior.

7. Solution Visualizations

The following landscape pages show the best selected hospital sites for each lambda value. Red X markers are selected hospitals, blue dots are population points, and gray dots are candidate sites not selected in the final solution. The figures are placed in landscape format to keep the maps readable in the PDF.

Lambda = 1

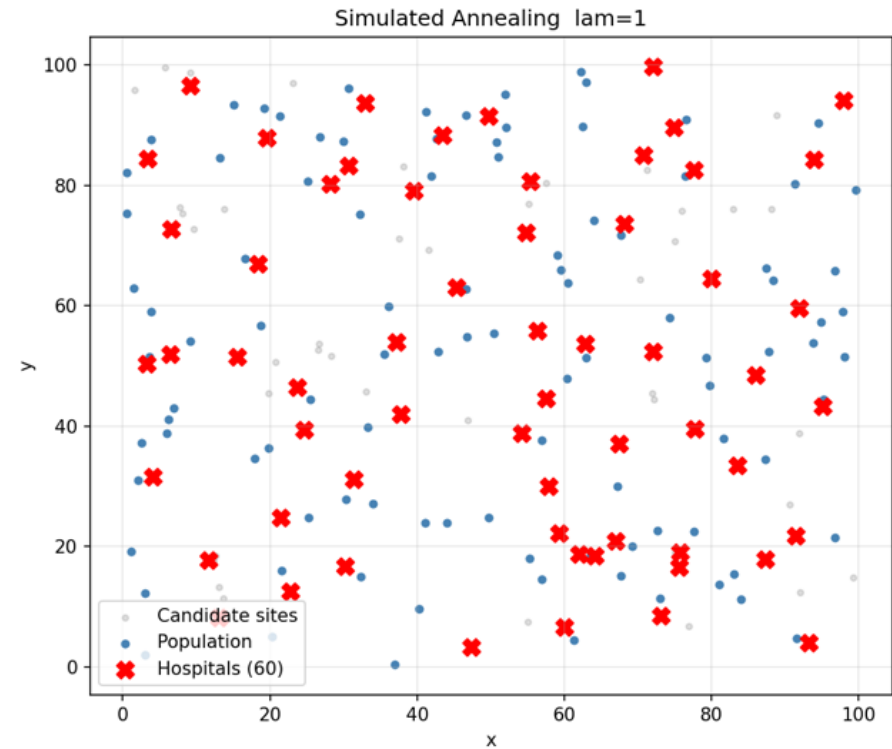
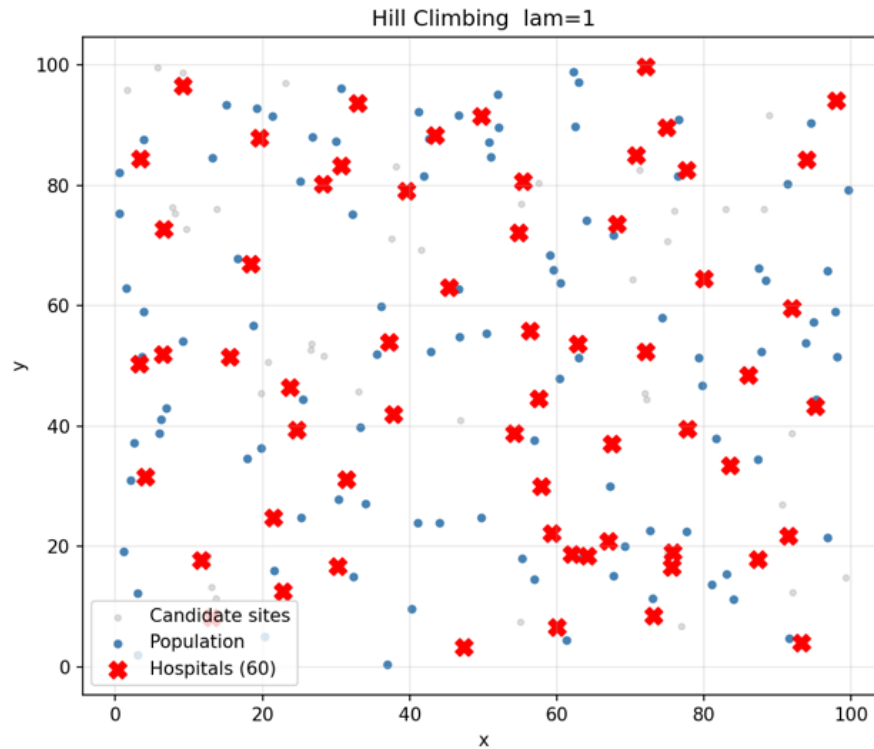


Figure 5: Best solutions for lambda = 1 using Hill Climbing and Simulated Annealing.

When lambda is low, hospitals are cheap, so both algorithms select many hospitals. The result is dense coverage across the space and the lowest travel distance.

Lambda = 10

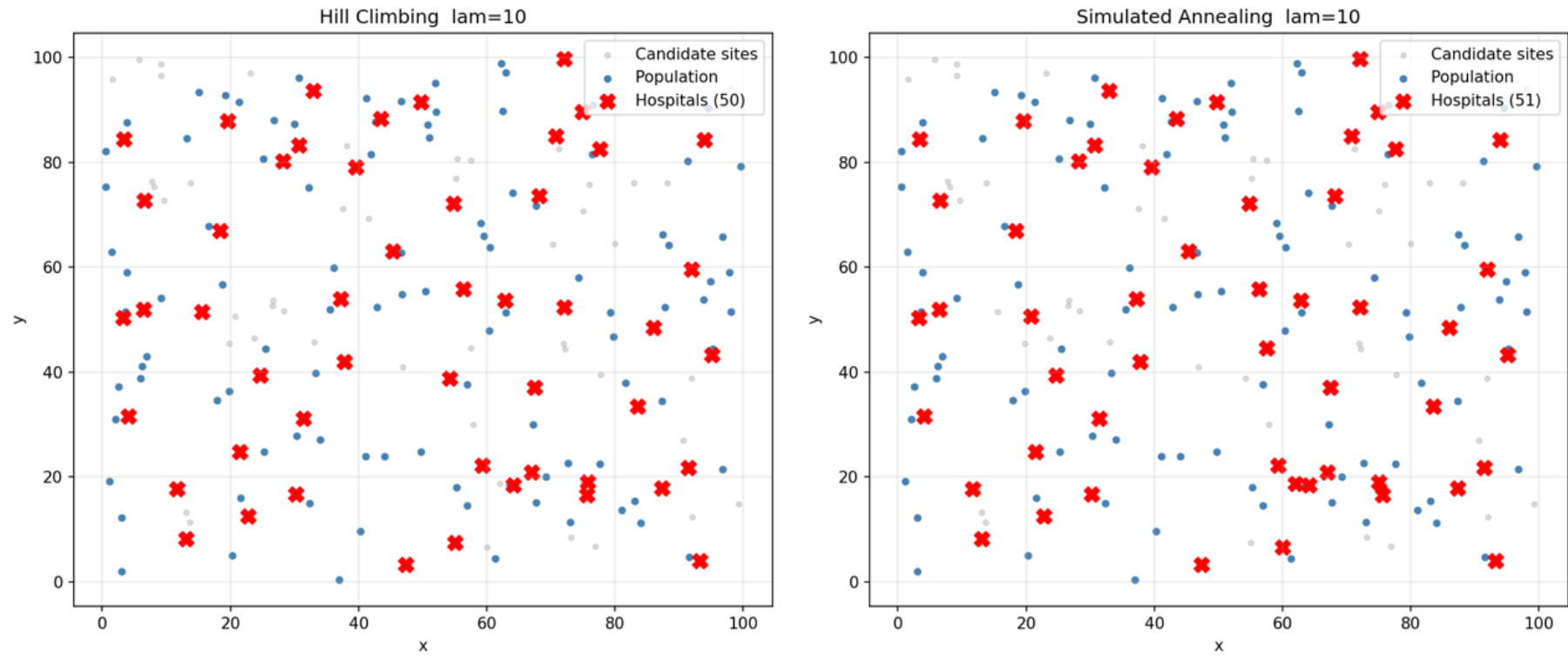


Figure 6: Best solutions for $\lambda = 10$ using Hill Climbing and Simulated Annealing.

At $\lambda = 10$, the algorithms still keep many hospitals, but begin removing locations that do not significantly reduce travel.

Lambda = 50

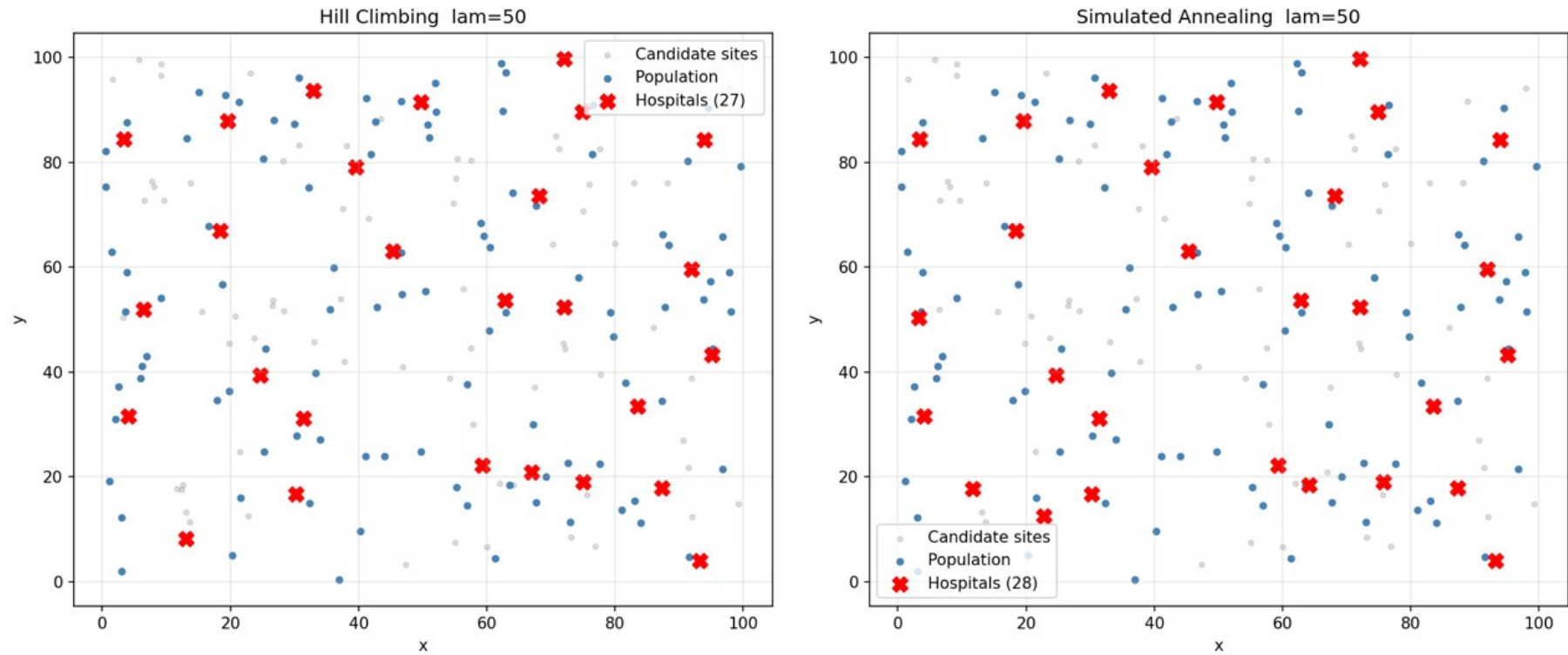


Figure 7: Best solutions for $\lambda = 50$ using Hill Climbing and Simulated Annealing.

At $\lambda = 50$, the solution becomes much sparser. This setting clearly shows the trade-off point: travel distance is higher, but the number of hospitals is reduced by more than half compared with $\lambda = 1$.

Lambda = 100

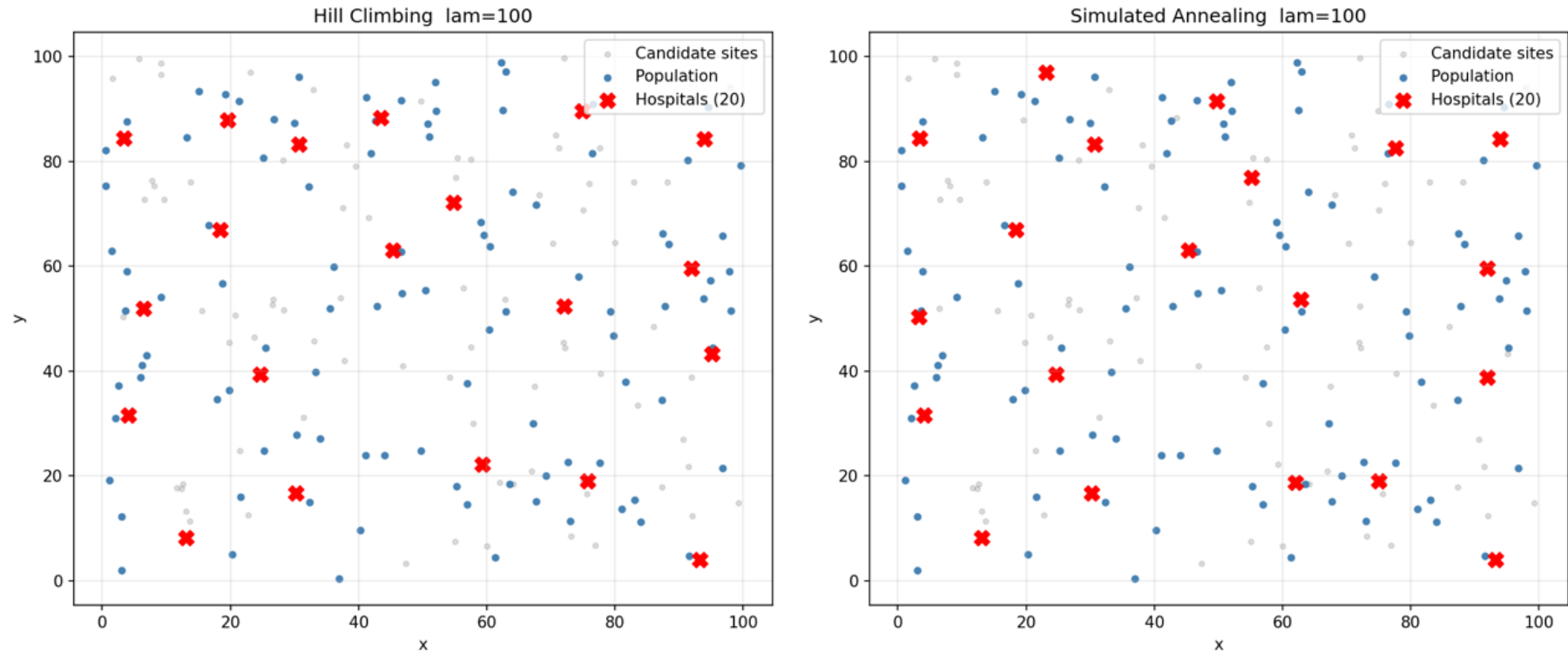


Figure 8: Best solutions for $\lambda = 100$ using Hill Climbing and Simulated Annealing.

At $\lambda = 100$, only about 20 hospitals remain. The hospitals become more central and spread out, which lowers the construction penalty but increases average travel distance.

7.1 Observed Lambda Pattern

Lambda	Observed behavior
1	Dense coverage; both algorithms select 60 hospitals and achieve the same cost.
10	Moderate penalty; around 50 hospitals remain and travel distance is still low.
50	Strong penalty; solutions select around 27-28 hospitals and travel distance rises.
100	Very strong penalty; around 20 hospitals remain and coverage becomes sparse.

Table 7: Summary of the λ effect in the visual solutions.

8. Parameter Tuning

The project requires studying at least one parameter for each algorithm. Tuning was performed at $\lambda = 50$ using 5 runs for each setting. For Hill Climbing, the tested parameter is the initial hospital selection rate. For Simulated Annealing, the tested parameter is the cooling rate α .

8.1 Hill Climbing: Initialization Rate

Variant	Mean Cost	Best Cost	Variance	Hosp.	Avg. Travel	Time(s)
HC init=5%	5100.33	5073.15	362.76	27.0	6.79	0.099
HC init=15%	5128.72	5083.81	1477.57	27.2	6.83	0.102
HC init=30%	5149.31	5103.18	1222.57	27.4	6.85	0.115

Table 8: Hill Climbing initialization-rate tuning at $\lambda = 50$.

The 5% initialization rate gives the lowest mean cost and lowest variance. A sparse initial solution appears more suitable here because the search can add useful hospitals rather than starting with many unnecessary hospitals and relying on removal.

8.2 Simulated Annealing: Cooling Rate Alpha

Variant	Mean Cost	Best Cost	Variance	Hosp.	Avg. Travel	Time(s)
SA $\alpha=0.9$	5176.69	5110.53	2651.92	27.8	6.86	1.041
SA $\alpha=0.95$	5203.49	5124.85	3081.72	28.2	6.87	1.064
SA $\alpha=0.99$	5200.77	5164.23	1426.84	28.4	6.85	1.004

Table 9: Simulated Annealing cooling-rate tuning at $\lambda = 50$.

Within the fixed budget, $\alpha = 0.90$ gives the best mean cost, while $\alpha = 0.99$ gives lower variance. A slower cooling rate explores more smoothly, but under a limited budget it may remain too exploratory for too long.

9. Discussion

Aspect	Hill Climbing	Simulated Annealing
Search behavior	Greedy best-improvement	Randomized exploration with probabilistic acceptance
Use of budget	Often stops early at local optimum	Uses full evaluation budget
Runtime in results	Lower runtime	Higher runtime
Stability	Lower variance	Higher variance
Main risk	Local optimum	Parameter sensitivity and stochastic variation
Best use case here	Fast, stable optimization on this instance	Alternative exploratory method when escaping local optima is important

Table 10: Qualitative comparison of the two algorithms.

9.1 Solution Quality

Hill Climbing achieves equal or better mean cost than Simulated Annealing for all tested λ values. The difference is small at $\lambda = 10$ but becomes clearer at $\lambda = 50$ and $\lambda = 100$. This suggests that, for this generated instance and this evaluation budget, best-improvement local search is more efficient than random-neighbor annealing.

9.2 Runtime and Evaluations

Hill Climbing is faster in wall-clock time because it stops when no improving neighbor exists. It uses fewer evaluations than the full budget in all main settings. Simulated Annealing uses the full 20,000 evaluations, which gives it more opportunities to explore but also increases runtime.

The evaluation counter is also useful in the demo because it explains why HC may report fewer evaluations than SA: HC can terminate early at a local optimum, while SA continues sampling neighbors until the budget is reached. Skipped invalid moves, such as trying to remove the only selected hospital, are not counted as evaluated neighbors because no objective function call is made for them.

9.3 Stability

The variance results show that Hill Climbing is more stable. Simulated Annealing intentionally introduces randomness to escape local optima, but this also makes its final result less predictable. This is especially visible at high λ values, where small changes in the selected subset can significantly change the total cost.

9.4 Effect of Lambda

The trend across lambda values confirms that the objective function behaves correctly. Increasing lambda reduces the number of hospitals and increases travel distance. This means the implementation correctly captures the intended trade-off between accessibility and construction cost.

10. Limitations and Possible Improvements

- Only one randomly generated problem instance was used. More instances would make the comparison stronger.
- The population distribution is uniform. Real population data is usually clustered, which may change the best hospital locations.
- Hill Climbing can still get stuck in local optima. Random restarts could improve its robustness.
- Simulated Annealing may need a better temperature schedule or adaptive cooling to use the evaluation budget more effectively.
- A Genetic Algorithm could be added as a third method, especially because it explores multiple candidate solutions at the same time.

11. Conclusion

This project implemented and compared Hill Climbing and Simulated Annealing for optimizing hospital locations. The experiments show that both algorithms respond correctly to the lambda parameter: higher lambda leads to fewer hospitals and higher average travel distance.

Under the shared evaluation budget, Hill Climbing produced lower or equal mean cost, lower variance, and much shorter runtime than Simulated Annealing. Simulated Annealing remains useful because it can accept worse moves and escape some local optima, but in this experiment its stochastic behavior caused higher variance and slower execution.

The parameter-tuning experiment showed that a 5% initialization rate performed best for Hill Climbing, while $\alpha = 0.90$ performed best for Simulated Annealing within the fixed 20,000-evaluation budget. Overall, Hill Climbing was the stronger method for this specific problem setup, while Simulated Annealing would benefit from further schedule tuning or a larger evaluation budget.